Санкт-Петербургский государственный университет

Аносов Павел Игоревич

Выпускная квалификационная работа

Реализация алгоритма верификации динамических запрещающих ограничений в проекте “Desbordante”

Уровень образования: бакалавриат

Направление *02.03.03 «Математическое обеспечение и администрирование информационных систем»*

Основная образовательная программа *СВ.5162.2022 «Технологии программирования»*

Научный руководитель:
к.ф.-м.н., старший научный сотрудник кафедры информационно-аналитических систем
Е. Г. Михайлова

Консультант:
ассистент кафедры информационно-аналитических систем Г. А. Чернышев

Рецензент:
Инженер-программист, ООО «КНС групп» Смирнов Александр Андреевич

Санкт-Петербург
2026

Saint Petersburg State University

Anosov Pavel

Bachelor's Thesis

Incremental verification of denial constraints in Desbordante

Education level: bachelor

Speciality *02.03.03 "Software and Administration of Information Systems"*

Programme *CB.5162.2022 "Programming Technologies"*

Scientific supervisor:
C.Sc., Senior researcher E. G. Mikhailova

Consultant:
Assistant G. A. Chernishev

Reviewer:
Software Engineer, LLC "KNS group" A. A. Smirnov

Saint Petersburg
2026

Оглавление

Введение **4**

1. Постановка задачи **5**

2. Обзор предметной области **6**

2.1. Статические подходы 6

2.2. Динамические подходы 7

2.3. Обоснование выбора алгоритма 8

2.4. Основные определения 9

2.5. Пример 10

3. Алгоритм **11**

3.1. Структуры данных 11

3.2. Статический анализ 19

3.3. Динамический анализ 20

4. Реализация **26**

4.1. LT-tree 26

4.2. Динамические таблицы 28

4.3. Индекс 31

4.4. Алгоритм `Weever` 34

4.5. Python-биндинги 38

5. Эксперименты **39**

5.1. Исследовательские вопросы и метрики 39

5.2. Условия эксперимента 39

Заключение **45**

Список литературы **46**

Введение

Desbordante [5] представляет собой высокопроизводительный профилировщик данных, предназначенный для обнаружения скрытых зависимостей и паттернов, которые трудно выявить стандартными методами профилирования.

Одним из ключевых классов таких зависимостей являются запрещающие ограничения (Denial Constraints, DC) [4]. Запрещающие ограничения (ЗО) задают условия на комбинации значений атрибутов, которые не должны встречаться в корректных данных. Диапазон выразительности ЗО широк: от унарных предикатов, относящихся к одной записи (например, неотрицательность зарплаты), до сложных условий, которые отражают отношения между парами записей в таблице. Выявление нарушений ЗО позволяет обнаруживать ошибки, опечатки и семантические несоответствия в данных.

Предыдущие работы [20, 21, 22] были посвящены верификации ЗО и приближённых ЗО, а также поиску нарушающих кортежей в статических данных. Однако при реальном использовании профилировщиков данные часто обновляются: в таблицу добавляются новые записи или удаляются существующие. Повторный запуск верификации на полном наборе данных при каждом изменении является вычислительно неэффективным. В связи с этим актуальной задачей становится разработка методов динамической верификации, позволяющих верифицировать ЗО и находить нарушения без перезапуска алгоритма при модификации набора данных.

В данной работе продолжается развитие инфраструктуры для работы с запрещающими ограничениями в рамках системы Desbordante. Основной целью является реализация верификатора запрещающих ограничений, поддерживающего динамическое обновление данных. Данное решение позволит эффективно отслеживать соблюдение ЗО в условиях, когда в процессе верификации ЗО происходит добавление или удаление кортежей.

1. Постановка задачи

Данная работа является идейным продолжением работ [20, 21, 22], в которых отсутствовала динамическая верификация запрещающих ограничений.

Целью настоящей работы является расширение имеющейся инфраструктуры, а также реализация самого алгоритма динамической верификации ЗО. Для достижения этой цели были поставлены следующие задачи:

1. Провести обзор существующих решений;
2. Реализовать алгоритм `Weever` для динамической верификации ЗО:
   (a) Реализовать поддержку динамического обновления таблиц с выводом типов;
   (b) Реализовать структуру LT-tree для индексирования динамической таблицы;
3. Сделать возможным использование динамической верификации в `Python`;
4. Провести экспериментальное исследование алгоритма.

2. Обзор предметной области

В области запрещающих ограничений выделяют две связанные, но различные задачи. *Поиск* зависимостей состоит в обнаружении множества всех запрещающих ограничений (Denial Constraints, DC), которые удерживаются на конкретном наборе данных. *Верификация* зависимостей, напротив, работает с заранее заданным ограничением и отвечает на вопрос, выполняется ли оно на данных, а в более общей постановке — какие именно пары кортежей его нарушают. Поиск и верификация также применимы и к другим видам зависимостей — функциональным зависимостям, зависимостям порядка или включения.

Настоящая работа посвящена задаче верификации, а именно — задаче динамической верификации. Задача верификации активно исследуется в течение последнего десятилетия. Существующие подходы к её решению можно условно разделить на две категории: *статические* верификаторы, которые проверяют удерживается ли запрещающее ограничение (ЗО) на исходном наборе данных, и *динамические* (инкрементальные) — способные эффективно обновлять множество нарушений при изменении данных без полного перезапуска. В оставшейся части раздела кратко рассматриваются ключевые представители обеих категорий и обосновывается выбор алгоритма, лежащего в основе данной работы.

2.1. Статические подходы

Facet. Алгоритм `Facet`, описанный в работе [14] использует конвейерный подход (refinement pipeline): предикаты ЗО обрабатываются последовательно, и на каждом шаге множество пар-кандидатов сужается за счёт отбрасывания тех пар, которые не удовлетворяют текущему предикату. Результатом работы алгоритма является *упорядоченное* множество пар кортежей, нарушающих заданное ЗО.

Rapidash и DCVerifier. `Rapidash` [15] использует другой подход: задача верификации ЗО сводится к задаче ортогонального диапазон-

ного поиска (orthogonal range search) в kd-дереве. Описанный авторами подход позволяет лишь определять, удерживается ли ЗО на наборе данных или нет. Нахождение множества нарушений в таблице не поддерживается. В рамках работы [22] алгоритм `Rapidash` был модифицирован таким образом, чтобы можно было не только проверять сам факт того, удерживается ли ЗО, но и находить нарушения. Полученный алгоритм был назван `DCVerifier`.

2.2. Динамические подходы

Идея инкрементальной обработки заключается в том, чтобы при изменении данных пересчитывать только ту часть результата, которая действительно изменилась. Подобный подход используется для разных классов зависимостей. Одной из первых работ в этом направлении стала статья "Incremental discovery of functional dependencies using partitions" [18], в которой авторы описывают алгоритм, обнаруживающий функциональные зависимости (ФЗ) при добавлении новых записей. Эта идея получила развитие в алгоритме `DynFD` [6], поддерживающем актуальное множество ФЗ при пакетных модификациях и уже реализуемом в `Desbordante`. Для уникальных комбинаций столбцов (UCC) аналогичную задачу решает алгоритм `Swan` [1], эффективно отслеживающий UCC при добавлении или удалении данных. Также инкрементальные методы предложены и для других классов зависимостей: порядковых [7, 10] и включения [16]. Все перечисленные работы относятся к задаче динамического *поиска* множества зависимостей при изменяющихся данных.

В контексте запрещающих ограничений динамический подход развивался преимущественно в направлении динамического *поиска* ЗО (поиска множества ЗО, которые выполняются на изменяющемся наборе данных) в работах [9, 13], а также инкрементального *восстановления* данных, нарушающих заданные ЗО [2]. Эти задачи отличаются от рассматриваемой в настоящей работе: первая отвечает на вопрос, *какие* ЗО справедливы для данных, а вторая направлена на исправление нару-

шений в обновляемых данных. Задаче динамической *верификации* ЗО на сегодняшний день посвящена единственная работа — “Incremental Detection of Denial Constraint Violations” [11], где авторы предложили алгоритм `Weever`.

Weever. `Weever` [11] — первый инкрементальный верификатор ЗО, предложенный в 2024 году. В отличие от работ по обнаружению и восстановлению, Weever сосредоточен именно на задаче поддержания актуального множества нарушений: при вставке, обновлении или удалении кортежей алгоритм обновляет это множество, не выполняя полного перезапуска. Как и статический алгоритм `Facet` [14], он предлагает конвейерный подход с последовательным сужением множества пар-кандидатов на нарушения, но лишь для обновляемых данных.

2.3. Обоснование выбора алгоритма

В рамках проекта Desbordante ранее были реализованы статический верификатор ЗО [20], верификатор приближённых ЗО [21], а также поиск нарушений в ЗО [22]. Что касается динамических подходов, в проекте присутствовал только инкрементальный поиск функциональных зависимостей. Динамическая верификация ЗО — более общего и выразительного класса ограничений, — отсутствовала. Настоящая работа расширяет применение динамических возможностей в `Desbordante` на класс ЗО и тем самым закрывает этот пробел.

Как отмечено выше, Weever является единственным известным алгоритмом, решающим задачу инкрементальной верификации ЗО. Он поддерживает все возможные виды динамических операций над данными: вставки, удаления и обновления кортежей. Также он предоставляет возможность отслеживать актуальные пары нарушений при изменении данных. По этим причинам именно Weever был выбран в качестве основы для реализации в данной работе.

Подробное описание теоретической части алгоритма приводится в разделе 3, а реализация — в разделе 4.4.

2.4. Основные определения

Перед тем, как приступить к описанию алгоритма проверки динамических ЗО, необходимо ввести основные определения и понятия. Все определения взяты из статьи [15].

Таблица 1: Основные понятия

Понятие	Значение
R	таблица с исходными данными
$vars(R)$	конечное множество атрибутов таблицы
$\|R\|$	количество строк в R
s, t	кортежи в R
$t.A$	значение атрибута A в кортеже t
op	бинарный оператор, $op \in \{=\ ,\ \neq\ ,\ \geq\ ,\ >\ ,\ \leq\ ,\ <\}$
предикат p	выражение вида $s.A\ op\ t.B$, $t,\ s \in R$, $A, B \in vars(R)$
ЗО φ	$\forall t, s \in R,\ t \neq s : \neg(p_1 \wedge \cdots \wedge p_m)$

Определение 1 *Пара кортежей (s, t) будет считаться нарушением (violation), если все предикаты в φ верны $\Leftrightarrow (s,\ t)$ violates φ.*

Определение 2 *ЗО $\varphi : \forall t, s \in R,\ t \neq s : \neg(p_1 \wedge \cdots \wedge p_m)$ называется* симметричным*, если для любой пары кортежей $(t, s), t \neq s$ выполняется:*

$$(t, s)\ violates\ \varphi \Leftrightarrow (s, t)\ violates\ \varphi.$$

В данной работе будем считать, что нарушения упорядочены, то есть порядок следования кортежей имеет значение. Это особенно важно для несимметричных запрещающих ограничений, где пара (s, t) — может быть нарушением, а (t, s) — нет.

Определение 3 *φ удержимается (holds) на R, если нет нарушений.*

2.5. Пример

В таблице 2 из оригинальной статьи [11] приведен набор данных, в котором указаны данные о доставках посылок. Вот некоторые правила, которые выполняются на данном наборе данных: (1) *package code* — потенциальный ключ и (2) $origin, destination \rightarrow distance$ — функциональная зависимость (расстояние между двумя географическими точками должно быть одинаковым независимо от направления движения).

Таблица 2: Отслеживаемые посылки

	package code	origin	destination	distance $[km]$	volume $[m^3]$	postage [$]
	0	London	Cape Town	9700	8	100
	1	Cape Town	Lima	9700	18	50
	2	London	New Delhi	6600	30	25
	3	Cape Town	London	9700	45	200
	4	New Delhi	London	6700	45	50
insert	5	Mexico City	Monaco	9700	18	10
delete	4	New Delhi	London	6700	45	50

Все вышеперечисленные правила можно записать в терминах запрещающих ограничений следующим образом:

1. $\varphi_1 : \forall s, t \in R, s \neq t : \neg(t.\text{package code} = s.\text{package code})$

2. $\varphi_2 : \forall s, t \in R, s \neq t : \neg\big(s.\text{origin} = t.\text{destination} \wedge s.\text{destination} = t.\text{origin} \wedge s.\text{distance} \neq t.\text{distance}\big)$

3. Алгоритм

Описанный авторами алгоритм позволяет находить нарушения ЗО на заданном наборе данных динамически: то есть при обновлении данных нет необходимости заново запускать алгоритм, чтобы понять, какие пары записей нарушают зависимость.

В предыдущих исследованиях проверка ЗО и поиск нарушений, как правило, выполнялись статически, что требовало повторного анализа всего набора данных при каждом обновлении. В отличие от этого, авторы предлагают динамический подход, основанный на последовательном сужении множества кандидатов, образующих нарушения.

Алгоритм состоит из двух частей: первая часть включает в себя статический анализ исходного набора данных, в ходе которого определяются все пары нарушений, присутствующие в исходном наборе данных. Второй этап алгоритма предназначен для верификации ЗО при обновлении исходного набора данных.

3.1. Структуры данных

В данном разделе будут описаны все необходимые для реализации алгоритма структуры данных, введенные авторами статьи [11].

3.1.1. Roaring bitmap

В исходной статье [11] авторы используют сжатые битовые множества — структуру для эффективного выполнения операций над множествами: добавления, удаления, пересечения и объединения элементов.

Roaring bitmap [12] представляет собой улучшенную реализацию сжатых битовых векторов, предназначенную для хранения множеств целых чисел. Она разбивает пространство значений на блоки фиксированного размера и для каждого блока динамически выбирает наиболее подходящее внутреннее представление. В зависимости от плотности данных используются различные контейнеры для их хранения. Такое

адаптивное представление позволяет эффективно балансировать между использованием памяти и скоростью выполнения операций.

Структура поддерживает основные операции над множествами: пересечение, объединение, разность, а также вставку и удаление элементов. Все операции выполняются поконтейнерно, то есть обрабатываются только те блоки, которые присутствуют в соответствующих множествах, что позволяет избежать перебора всего диапазона значений. Временная сложность добавления и удаления составляет амортизированное время $O(1)$. Для бинарных операций же временная сложность различается в зависимости от используемого при хранении типа контейнера.

Сравним Roaring bitmap с двумя классическими альтернативами. Традиционные хеш-множества обеспечивают амортизированное время $O(1)$ для операций вставки, удаления и проверки принадлежности. Однако для выполнения операций пересечения и объединения они требуют последовательного перебора элементов за $O(\min(N, M))$, где N, M — размеры этих множеств. Roaring bitmap, напротив, опирается на побитовые операции, что существенно ускоряет вычисления: при плотных или сплошных диапазонах значений используемые контейнеры поддерживают быстрые битовые операции.

Динамические битовые массивы (*dynamic bitset*) потребляют количество памяти, строго пропорциональное размеру U множества принимаемых значений — $O(U)$, и становятся крайне неэффективными при разреженных данных. В то же время `Roaring` адаптивно выбирает формат хранения, что позволяет ему сохранять компактность при разреженных множествах и высокую скорость операций при плотных данных. Благодаря такому подходу `Roaring bitmap` устраняет главные недостатки обоих аналогов, предлагая оптимальное сочетание скорости операций над множествами и экономии памяти.

3.1.2. LT-tree

Авторы вводят новую структуру данных Less-Than-Tree (LT-tree), предназначенную для быстрого поиска всех индексов значений, меньших данного. Она используется для эффективной поддержки предикатов с операторами неравенства вида $t.A\ op\ s.B$, $op \in \{<, \leq, >, \geq\}$ при верификации запрещающих ограничений.

В исходной статье LT-tree реализовано на основе красно-чёрного дерева [8]. Однако данная структура может быть построена на основе любого самобалансирующегося дерева поиска. Поскольку конкретные правила балансировки не влияют на описываемую логику поддержания множеств индексов, в дальнейшем мы не будем детализировать механизм перестроения узлов, ограничившись общими принципами работы самобалансирующихся структур.

Структура узла LT-tree. LT-tree реализуется на основе самобалансирующегося бинарного дерева поиска. Каждый узел дерева хранит следующие компоненты:

1. v — ключ узла, представляющий собой конкретное значение индексируемого атрибута;

2. $T(v)$ — множество идентификаторов записей, в которых индексируемый атрибут принимает значение v;

3. $LT(v)$ — множество идентификаторов всех записей, значения индексируемого атрибута которых содержатся в левом поддереве узла.

Поскольку множества $T(v),\ LT(v)$ могут содержать большое количество идентификаторов, их необходимо быстро объединять и пересекать при добавлении или удалении значений в `LT-tree`. В связи с этим авторы статьи предлагают вместо хранения самих идентификаторов в чистом виде использовать сжатые множества, которые обсуждались выше. Они занимают значительно меньше памяти, а также поддерживают быстрые операции пересечения и объединения. Таким образом,

каждый узел хранит информацию обо всех значениях, меньших или равных текущему ключу, что позволяет эффективно выполнять запросы по поиску всех ключей, меньших или равных искомому значению.

Вставка. Обход LT-tree при поиске осуществляется аналогично стандартному бинарному дереву поиска: при посещении узла искомое значение сравнивается с ключом узла. Если искомое значение больше ключа — алгоритм переходит к правому поддереву, если меньше — к левому, а при равенстве поиск завершается. Однако для поддержки быстрого поиска индексов меньших значений необходимо расширить стандартные процедуры вставки и удаления, добавив логику поддержания множеств $T(v)$ и $LT(v)$. Рассмотрим процедуру добавления подробнее. Пусть t — очередной обрабатываемый кортеж, A — индексируемый атрибут, $t.A$ — значение атрибута в обрабатываемом кортеже, а $id(t)$ — уникальный идентификатор кортежа. Ниже представлен алгоритм вставки значения $v = t.A$ в LT-tree:

1. Ищем узел с ключом $t.A$ в дереве, начиная с корня;
2. Если $t.A$ больше ключа в текущем узле — переходим к правому поддереву, если меньше — к левому;
3. При каждом переходе в левое поддерево текущего узла идентификатор $id(t)$ добавляется в множество $LT(v)$ текущего узла, так как значение $t.A$ меньше ключа v текущего узла;
4. Если узел с ключом $t.A$ найден, $id(t)$ добавляется в $T(v)$; иначе создаётся новый узел-лист с полями:
   (a) $v = t.A$,
   (b) $T(v) = \{id(t)\}$,
   (c) $LT(v) = \varnothing$;
5. При необходимости выполняется балансировка дерева в соответствии с правилами самобалансирующегося дерева.

Построение LT-tree осуществляется путём последовательной вставки значений атрибутов всех кортежей в сбалансированное дерево поиска. Для каждого кортежа t и индексируемого атрибута A выполняется вставка пары $(t.A,\ id(t))$, где $id(t)$ — идентификатор кортежа.

Сложность вставки: $O(\log^2 d)$, где d — количество различных значений в индексируемом столбце. Для вставки значения в дерево необходимо посетить $O(\log d)$ узлов, как в обычном бинарном дереве. Также логарифмическое количество времени требуется для операции добавления значения в сжатое множество идентификаторов [12].

Более подробно доказательства корректности оценок временной сложности для операций LT-tree можно найти в оригинальной работе [11].

Балансировка. При выполнении поворотов (левого или правого) в процессе балансировки дерева необходимо корректировать множества LT затронутых узлов. Пусть выполняется поворот между узлами x и y. Тогда:

1. для правого поворота множество $LT(y)$ обновляется путём удаления из него элементов $T(x) \cup LT(x)$;

2. для левого поворота множество $LT(y)$ формируется как объединение $T(x) \cup LT(x)$.

Пример. Рассмотрим вставку в LT-tree на примере таблицы 2 и индексируемого атрибута *postage*. Изначально в дереве 1 содержатся первые пять кортежей $(t_0, \ldots, t_4)$, затем выполняется вставка кортежа t_5 со значением $postage = 10$.

В дереве пять различных значений: $\{25, 50, 100, 200\}$ (значение 50 встречается дважды — у t_1 и t_4). После последовательной вставки этих кортежей и балансировки получаем дерево, в котором корнем является узел со значением 100.

Выполним вставку кортежа t_5 согласно алгоритму, описанному выше:

Шаг 1. Начинаем обход с корня (узел с ключом 100). $10 < 100 \Rightarrow$ переходим к левому поддереву. Так как мы выбрали левое поддерево,

добавляем t_5 в множество LT текущего узла:

$$LT(100) = \{t_1, t_2, t_4\} \cup \{t_5\} = \{t_1, t_2, t_4, t_5\}.$$

Шаг 2. Переходим в узел с ключом 50. Так как $10 < 50$, снова идём налево и обновляем множество $LT(50)$:

$$LT(50) = \{t_2\} \cup \{t_5\} = \{t_2, t_5\}.$$

Шаг 3. Переходим в узел с ключом 25. Так как $10 < 25$, идём налево и обновляем множество $LT(25)$:

$$LT(25) = \varnothing \cup \{t_5\} = \{t_5\}.$$

Шаг 4. Левый ребёнок узла с ключом 25 отсутствует. Значение 10 ещё не существует в дереве, поэтому создаём новый листовой узел:

$$\text{value} = 10, \quad T(10) = \{t_5\}, \quad LT(10) = \varnothing.$$

Шаг 5 (перебалансировка). После вставки дерево перестало удовлетворять свойствам сбалансированного дерева: высоты поддеревьев узла с ключом 50 различаются более чем на единицу, что создаёт несбалансированность в левом поддереве. Для восстановления баланса выполняется правый поворот относительно узла с ключом 50: узел с ключом 25 становится новым корнем поддерева, узел с ключом 50 — его правым ребёнком. Узел с ключом 10 остаётся на своём месте.

При этом необходимо обновить множество LT узла с ключом 50, так как его левое поддерево изменилось: ранее оно содержало узлы со значениями 25 и 10, а теперь левого поддерева у узла 50 нет. По правилу для правого поворота:

$$LT(50)_{\text{new}} = LT(50)_{\text{old}} \setminus \big(T(25) \cup LT(25)\big) = \{t_2, t_5\} \setminus \big(\{t_2\} \cup \{t_5\}\big) = \varnothing.$$

Множество $LT(25)$ не меняется, так как его левое поддерево (узел с ключом 10) осталось прежним. $LT(100)$ также не меняется, так как порядок узлов выше места поворота не затрагивается.

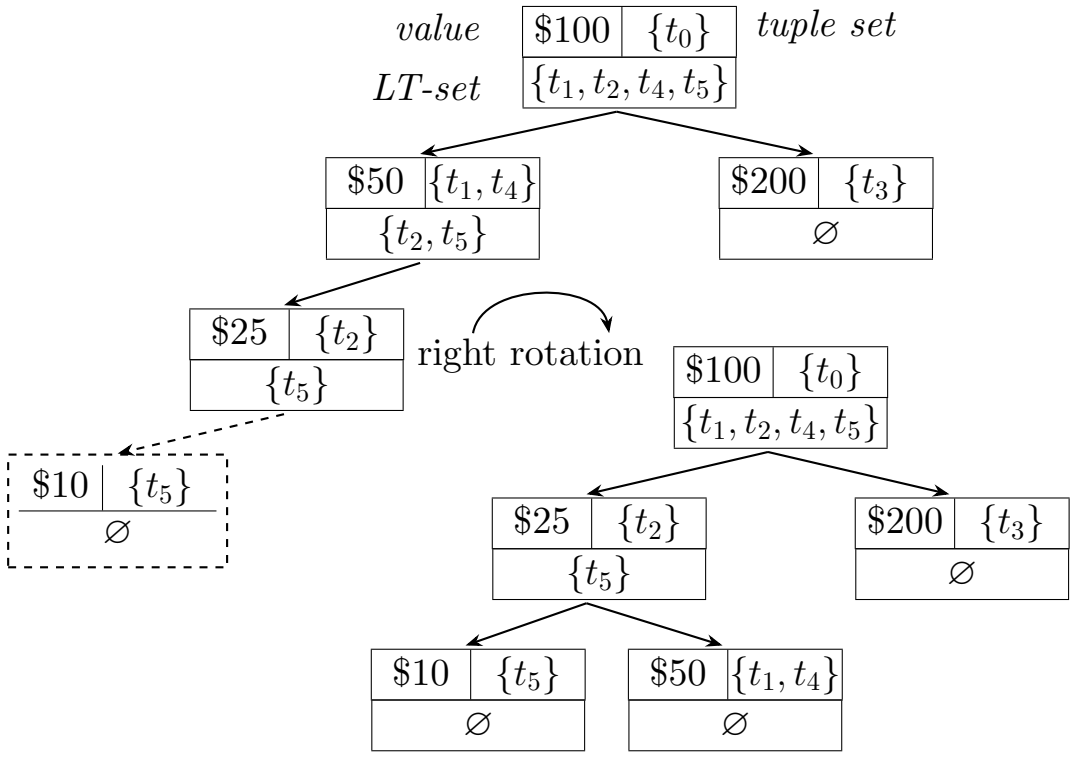


Рис. 1: Вставка нового значения в LT-tree: сверху — до балансировки, снизу — после правого поворота

Удаление. Алгоритм удаления кортежа из LT-tree аналогичен операции вставки, за исключением того, что мы удаляем идентификаторы из множеств $T(v)$ и $LT(v)$. Удаление включает в себя следующие шаги:

1. Поиск узла с ключом $v = t.A$.
2. При каждом переходе в левое поддерево текущего узла x идентификатор $id(t)$ удаляется из множества $LT(x)$.
3. Идентификатор $id(t)$ удаляется из множества $T(v)$.
4. Если после удаления $T(v)$ становится пустым, соответствующий узел удаляется из дерева.
5. Выполняется балансировка дерева по правилам самобалансирующегося дерева.

Сложность удаления: Аналогично операции вставки, удаление занимает $O(\log^2 d)$.

Поиск всех меньших значений. Основная операция LT-tree — поиск множества всех кортежей, имеющих значения меньше заданного v. LT-tree эффективно поддерживает такие операции за счёт использования множеств LT.

Algorithm 1: Поиск множества идентификаторов всех записей, в которых индексируемый атрибут имеет значения строго меньше искомого

Data: searchValue v

Result: `smallerTuples`

```
1 function FindAllSmaller()
2     x ← rootNode, smallerTuples ← ∅
3     while v ≠ x.value do
4         if v > x.value then
5             smallerTuples ← smallerTuples ∪ x.T ∪ x.LT
6             x ← x.rightChild
7         else
8             x ← x.leftChild
9     return smallerTuples
```

Поиск кортежей, значения индексируемого атрибута которых меньше или равны заданному v (операция `FindLessEqual`), организован аналогично описанному выше алгоритму. Отличие состоит лишь в том, что при нахождении узла, ключ которого совпадает с искомым значением v, множество $T(v)$ этого узла также включается в результат:

$$\texttt{smallerTuples} \leftarrow \texttt{smallerTuples} \cup x.T.$$

Эта операция используется при обработке предикатов с операторами $\leq$ и $\geq$ (см. раздел 3.3.3).

Сложность поиска: обход дерева выполняется за $O(\log d)$, а формирование результата — за $O(n)$ в худшем случае, где n — число кортежей в отношении.

Количество занимаемой памяти. Пусть n — количество кортежей, равномерно распределённых по d различным значениям. Согласс-

но утверждениям авторов, асимптотическая оценка занимаемой LT-tree памяти составляет $O(\log d \cdot n + d)$. По сравнению со стандартным деревом, занимающим $O(n+d)$ памяти, дополнительные накладные расходы можно считать допустимыми. На практике потребление памяти оказывается ещё менее значительным: множества идентификаторов кортежей хранятся в виде сжатых битовых множеств, поэтому фактическое потребление памяти остаётся довольно низким.

3.2. Статический анализ

Чтобы нарушить заданное запрещающее ограничение, пара кортежей (s, t) должна удовлетворять всем его предикатам. Поскольку ЗО представляет собой отрицание конъюнкции предикатов, невыполнение хотя бы одного предиката предотвращает нарушение.

Это свойство позволяет эффективно проверять запрещающие ограничения, рассматривая только те пары кортежей, которые удовлетворяют всем ранее обработанным предикатам. Таким образом, предикаты ЗО естественным образом образуют конвейер обработки. Каждый предикат получает множество кандидатов (пар кортежей), потенциально нарушающих ограничение, от предыдущего предиката и удаляет те пары, которые ему не удовлетворяют.

Множество кандидатов называется промежуточным (intermediate), а процесс его сокращения — уточнением (refinement), которое выполняется с использованием специализированных алгоритмов для каждого оператора.

Изначально множество кандидатов содержит все пары кортежей. Оно последовательно уточняется каждым предикатом из ЗО. На каждом шаге из промежуточного множества удаляются те пары, которые не удовлетворяют рассматриваемому предикату. То есть, если все предикаты будут верны — пара будет образовывать нарушение. После обработки всех предикатов в промежуточном множестве остаются лишь нарушения, а именно $\{(t_2, t_4), (t_4, t_2)\}$.

Нарушения, найденные на этом этапе, составляют основу для под-

Таблица 3: Процесс уточнения для ограничения φ_2

Шаг	Обработанный предикат	Промежуточное множество
1	$s.\text{origin} = t.\text{destination}$	$\{(t_0, t_3), (t_0, t_4), (t_1, t_0),$ $(t_2, t_3), (t_2, t_4), (t_3, t_0), (t_4, t_2)\}$
2	$s.\text{destination} = t.\text{origin}$	$\{(t_0, t_3), (t_2, t_4), (t_3, t_0), (t_4, t_2)\}$
3	$s.\text{distance} \neq t.\text{distance}$	$\{(t_2, t_4), (t_4, t_2)\}$

держания актуальности данных без повторного сканирования при работе динамической части алгоритма.

3.3. Динамический анализ

Авторы статьи предлагают поддержку добавления новых строк в конец таблицы и удаление существующих строк. Обновление уже существующих строк моделируется как удаление заданной строки и вставка новой строки с таким же идентификатором.

3.3.1. Добавление

В статическом обнаружении нарушений ЗО промежуточное множество состоит из пар кортежей — в инкрементальном сценарии используется другая структура. Поскольку каждое новое добавление может создать новые нарушения только с уже существующими кортежами, начальное множество кандидатов содержит все существующие кортежи.

Неявно новый кортеж образует потенциально нарушающую пару со всеми кортежами в промежуточном множестве. Так как ЗО определяется на парах кортежей, добавленный кортеж может выступать либо в роли s, либо в роли t при проверке.

Пусть $newT$ — вновь вставленный кортеж, а T — множество всех кортежей. Тогда создаются два независимых промежуточных множества:

$$I = \{t \in T \mid (newT,\ t) \text{ violates } \varphi\}, \quad I' = \{s \in T \mid (s,\ newT) \text{ violates } \varphi\}.$$

Авторы отмечают, что второе промежуточное множество не требуется для симметричных ЗО. Оба множества совпадают, если ни один предикат не использует два различных атрибута или не содержит неравенства. Кроме того, предикат может одновременно уточнять оба промежуточных множества, если оба операнда в предикате ссылаются на один и тот же атрибут.

Рассмотрим ЗО φ_3 и вставку кортежа t_5.

$$\varphi_3 : \forall s, t \in R, s \neq t : \neg\big(s.\text{distance} = t.\text{distance} \wedge s.\text{volume} < t.\text{volume} \wedge s.\text{postage} > t.\text{postage}\big)$$

Конвейер процедуры уточнения показан ниже. Изначально оба промежуточных множества содержат все кортежи. Так как одно из множеств в конце оказывается пустым, только пара (t_0, t_5) нарушает φ_3.

Таблица 4: Конвейер уточнения для φ_3 при вставке t_5

Шаг	Обработанный предикат	I	I'
1	$s.\text{distance} = t.\text{distance}$	$\{t_0, t_1, t_3\}$	$\{t_0, t_1, t_3\}$
2	$s.\text{volume} < t.\text{volume}$	$\{t_3\}$	$\{t_0\}$
3	$s.\text{postage} > t.\text{postage}$	$\emptyset$	$\{t_0\}$

Более подробно процесс уточнения промежуточных множеств рассмотрим в разделе 3.3.3.

3.3.2. Удаление

Поскольку удаления не могут создавать новые нарушения, нет необходимости обрабатывать процесс уточнения. Вместо этого удаления могут только устранять уже существующие нарушения, поэтому необходимо эффективно отслеживать их.

Например, удаление t_4 устраняет пары (t_2, t_4) и (t_4, t_2) из ранее найденных нарушений (см. таблицу 3) для φ_2.

Обработка удалений не требует процесса уточнения, поскольку не может привести к обнаружению новых нарушений. Достаточно исключить его из списка нарушений и удалить из индексов, построенных для

каждого столбца в ЗО.

Algorithm 2: Удаление кортежа

```
Data: удаляемый кортеж deletedTuple, индексы
Result: множество всех нарушений allViolations
1 function Delete()
2     tId ← GetKey(deletedTuple);
3     for violationMap ∈ allViolations do
4         DeleteFromViolationMap(violationMap, tId);
5     DeleteFromIndices(deletedTuple);
6     allIds ← allIds \ {tId};
7     return allViolations;
```

3.3.3. Уточнение промежуточных множеств

Обработка одного предиката требует двух шагов. Во-первых, мы обращаемся к соответствующему индексу для получения операнда. Во-вторых, промежуточные множества всех затронутых ЗО уточняются с использованием операнда. Операнд представляет собой набор идентификаторов кортежей, который используется для уточнения промежуточного множества, т.е. набора кандидатов на возможные нарушения.

Для эффективного получения операндов мы создаем индекс для каждого столбца, присутствующего в предикате: если столбец является частью предиката неравенства, мы используем LT-tree, иначе — простую хеш-таблицу.

Требуемый конкретный операнд зависит от типа предиката. Пусть $s.A\ op\ t.B$ — предикат, а $newT$ — вновь вставленный кортеж. Для предикатов с операторами $=$ или $\neq$ мы выбираем кортежи с тем же значением, что и у $newT$. Для предикатов с операторами $<, \leq, >, \geq$ мы ищем кортежи со значением меньше, чем у $newT$. Эта операция поддерживается LT-tree.

Промежуточное множество и операнд внутренне хранятся в виде сжатого битового вектора [12]. Таким образом, они позволяют выполнять быстрые операции над множествами, а также экономят простран-

ство.

Алгоритм применяет операнды к промежуточному множеству в зависимости от типа предиката. С одной стороны, он строит пересечение промежуточного множества и операнда для предикатов следующего типа

$$s.A = newT.B \quad s.A < newT.B \quad s.A \leq newT.B$$

Для этих предикатов операнд содержит те кортежи, которые удовлетворяют предикату. После пересечения с промежуточным множеством в нём остаются только те кортежи, которые потенциально образуют нарушение, а остальные отсеиваются. Например, предикат $s.\text{distance} = t.\text{distance}$ в φ_3 оставляет в промежуточном множестве только $\{t_0, t_1, t_3\}$ (см. таблицу 4). С другой стороны, алгоритм выполняет операцию разности множеств между промежуточным множеством и операндом для предикатов типа

$$s.A \neq newT.B \quad s.A > newT.B \quad s.A \geq newT.B$$

Здесь операнд содержит ровно те кортежи, которые не удовлетворяют предикату. Таким образом, мы удаляем их из промежуточного множества. Например, предикат $s.\text{distance} \neq t.\text{distance}$ в φ_2 удаляет $\{t_0\}$ из промежуточного множества в отличие от его аналога равенства.

Для одноколоночных предикатов, т.е. тех, что имеют вид $s.A$ *op* $t.A$, мы можем повторно использовать операнд для уточнения обоих промежуточных множеств. Предикаты равенства и неравенства рефлексивны, поэтому нам даже не нужно повторно применять операнд, до тех пор пока оба промежуточных множества остаются равными до предиката. Хотя предикаты неравенства требуют повторного применения операнда, нам не нужно снова обращаться к индексу. Каждый предикат неравенства может быть зеркально отражен, когда $newT$ становится s и t, например, аналогом $s.A < newT.A$ является $t.A > newT.A$. Таким образом, отличается только применение операнда, но не сам операнд.

3.3.4. Планирование предикатов ЗО

Порядок выполнения предикатов играет важную роль в определении производительности. В частности, мы можем прервать конвейер предикатов, если промежуточное множество пусто. Однако мы не можем точно предсказать вероятность того, что промежуточное множество окажется пустым после выполнения предиката. Поэтому мы классифицируем предикаты ЗО на следующие шесть групп и разрабатываем правила для обработки каждой из них отдельно. Мы обрабатываем предикаты в следующем порядке:

(1) одноколоночное равенство : $s.A = t.A$

(2) двуколоночное равенство : $s.A = t.B$

(3) одноколоночный оператор $\neq$: $s.A \neq t.A$

(4) двуколоночный оператор $\neq$: $s.A \neq t.B$

(5) одноколоночное неравенство : $s.A < t.A$

(6) двуколоночное неравенство : $s.A < t.B$

Выбранный порядок обусловлен тремя соображениями: стоимостью применения предиката, количеством задействованных столбцов и селективностью оператора.

Стоимость предиката. Обработка операторов $<, \leq, \geq, >$ обходится дороже всех остальных предикатов, так как требует больше операций над множествами. Таким образом, предикаты (5) и (6) выполняются в последнюю очередь.

Количество столбцов. Одноколоночные предикаты (1), (3) и (5) могут одновременно уточнять оба промежуточных множества I и I' за одно обращение к индексу, поскольку они обращаются к одному атрибуту, операнд для пары $(newT, t)$ совпадает с операндом для пары $(s, newT)$. Поэтому в каждой паре однотипных предикатов одноколоночные обрабатываются раньше двуколоночных.

Селективность оператора. Мы разделяем предикаты с оператором $=$ (группы (1) и (2)) и предикаты с оператором $\neq$ (группы (3) и (4)) по двум причинам. Во-первых, операторы $=$ являются более се-

лективными для любого столбца, в котором число уникальных значений больше двух: равенство выделяет одно значение из многих, тогда как неравенство — наоборот. Чем сильнее предикат сокращает промежуточное множество, тем выше шанс прервать конвейер на ранних шагах. Во-вторых, первоначальное построение промежуточного множества для равенств выполняется быстрее: можно просто скопировать операнд. Для предикатов с оператором $\neq$ необходимо скопировать все присутствующие идентификаторы кортежей и построить разность множеств с операндом.

Таким образом, выбранный порядок (1)–(6) сначала обрабатывает наиболее селективные и дешёвые предикаты с равенством, затем менее селективные с оператором $\neq$, и в последнюю очередь — наиболее дорогие предикаты неравенства, что в итоге сокращает объём вычислений за счёт более раннего прерывания конвейера на пустом промежуточном множестве.

4. Реализация

4.1. LT-tree

В оригинальной работе [11] структура `LT-tree` основывается на красно-чёрном дереве. В данной работе в качестве базовой сбалансированной структуры данных было использовано AVL-дерево [19], что не противоречит теоретическим оценкам алгоритма и не изменяет его асимптотических оценок сложности.

AVL-дерево обеспечивает более строгую балансировку по сравнению с красно-чёрным деревом, что приводит к меньшей высоте дерева в среднем и в худшем случае. Это свойство особенно важно для LT-tree, поскольку при выполнении основных операций (поиск и обновление агрегированных множеств меньших значений) стоимость операций пропорциональна длине пути от корня к листу. Однако более строгая балансировка также имеет свои минусы — она приводит к большему числу поворотов, вплоть до логарифма от высоты дерева, что плохо сказывается на операциях вставки и удаления. Таким образом, меньшая высота дерева позволяет снизить константные множители во временной сложности операций поиска, но увеличивает временные затраты при добавлении и удалении данных.

Поскольку AVL-дерево, как и красно-чёрное дерево, обеспечивает логарифмическую сложность основных операций поиска, вставки и удаления, выбор AVL-дерева не влияет на асимптотические оценки алгоритма верификации динамических запрещающих ограничений и сохраняет его теоретические гарантии корректности и эффективности.

Выбор AVL-дерева в качестве основы LT-tree обусловлен причинами, перечисленными выше, а также простотой реализации относительно красно-чёрного дерева.

Реализовать данную структуру поверх уже существующих решений из `boost` [3] (boost::intrusive::rbtree) или `STL` [17] (std::map, std::set) оказывается затруднительным, т.к. в узлы деревьев необходимо добавлять новые данные, а также иметь прямой контроль операций при балансi-

ровке.

Ниже представлена диаграмма 2 классов получившегося `LTTree`:

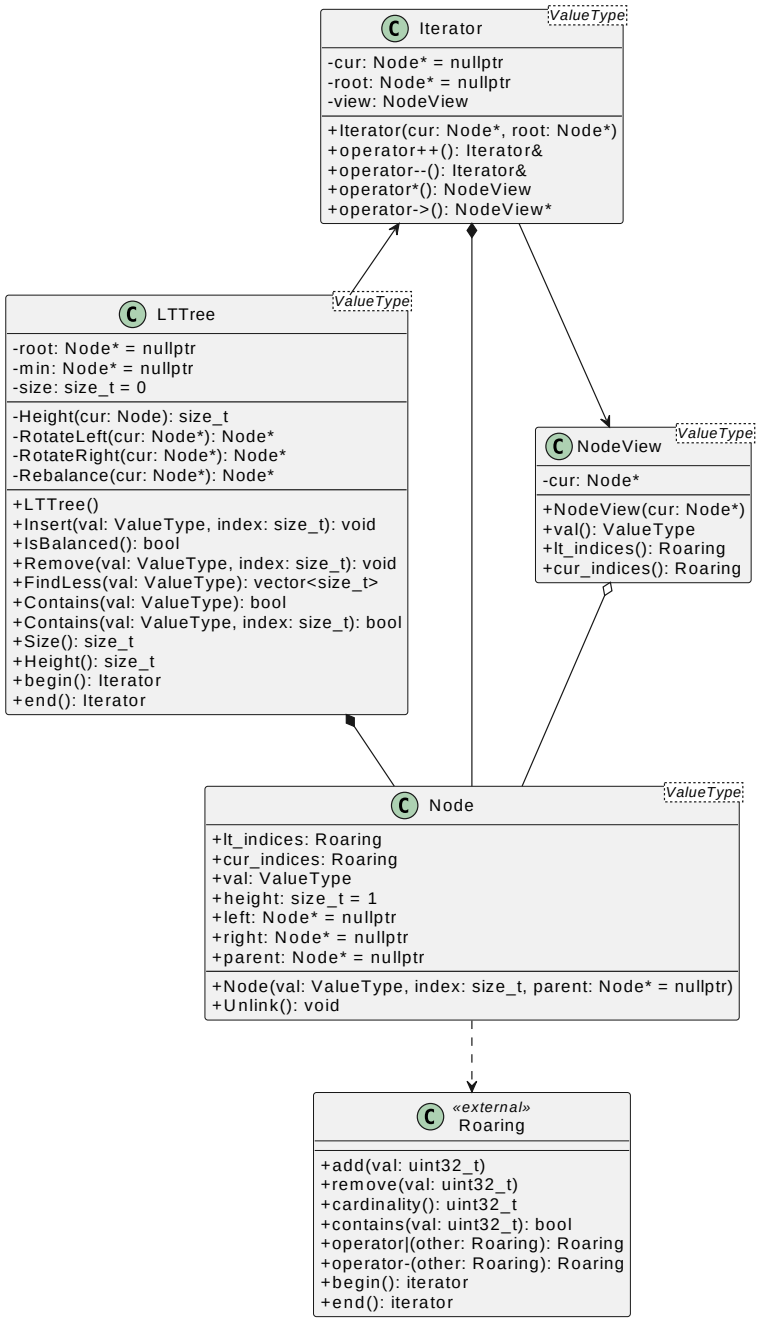


Рис. 2: Диаграмма класса `LTTree` и вспомогательных классов.

1. `LTTree` — основной класс, реализующий сбалансированное двоичное дерево поиска. Поддерживает операции вставки, удаления и поиска всех индексов элементов, меньших заданного значения;

2. `Iterator` — вложенный вспомогательный класс, использующийся

для обхода `LTTree` в симметричном (inorder) порядке от меньшего элемента к большему. Позволяет проходить дерево вперёд и назад, поддерживая концепцию `bidirectional_iterator`;

3. `Node` — внутренний узел дерева, хранит идентификаторы значений, равных и меньших текущего, а также ссылки на дочерние и родительский узлы;

4. `NodeView` — класс, использующийся для доступа к узлам дерева через `Iterator`. Предоставляет лишь ограниченный набор полей исходного узла `Node`, не раскрывая внутреннюю структуру дерева;

5. `Roaring` — класс, представляющий сжатое множество индексов. Позволяет добавлять, удалять элементы, а также выполнять битовые операции. В `LTTree` он используется для хранения и объединения индексов элементов, поддерживает добавление, удаление, проверку принадлежности, итерацию и эффективные битовые операции над множествами.

4.2. Динамические таблицы

Для работы алгоритма требуется поддерживать добавление новых записей в конец таблицы, а также удаление и обновление уже существующих записей. В системе Desbordante уже существовал класс динамической таблицы, поддерживающий эти операции, однако он не учитывал типы данных, которые хранятся в таблице — значения колонок интерпретировались как строки. Поддержка типов данных в таблице является неотъемлемой частью работы алгоритма, поскольку операнды в предикатах ЗО могут принимать целые, дробные, строковые или другие типы значений, и для каждого из таких типов операции сравнения выполняются по-разному.

В рамках данной работы существующий интерфейс динамической таблицы был расширен типизированной версией: добавлен интерфейс `ITypedDynamicTableData`. Также введён паттерн стратегии для способов обновления таблицы.

Ниже представлена диаграмма классов динамической таблицы:

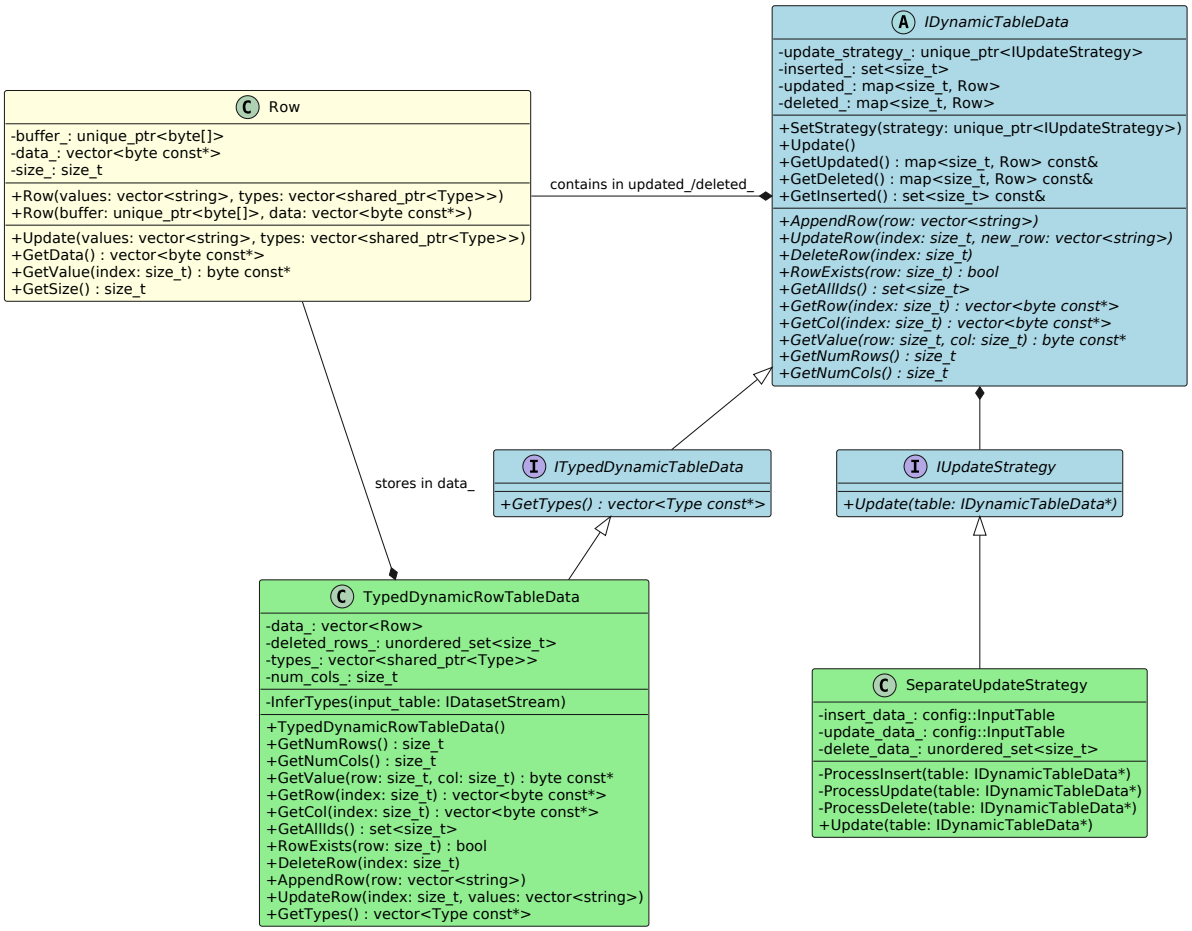


Рис. 3: Диаграмма класса динамической таблицы и вспомогательных классов.

1. `IDynamicTableData` — базовый интерфейс динамической таблицы. Определяет методы получения количества строк и столбцов, добавления, обновления и удаления строк, а также возможность установки стратегии обновления.

2. `ITypedDynamicTableData` — расширенный интерфейс динамической таблицы, поддерживающий вывод типов для колонок.

3. `Row` — класс, предоставляющий доступ к элементам строки.

4. `TypedDynamicRowTableData` — реализация динамической таблицы в строчном представлении. Поддерживает добавление, обновление и удаление строк, а также вывод типов для элементов строки.

5. `IUpdateStrategy` — интерфейс стратегии обновления динамической таблицы. Обеспечивает абстракцию для применения набора операций обновления к таблице через метод `Update`.

6. `SeparateUpdateStrategy` — конкретная стратегия обновления. Выполняет отдельные этапы: удаление строк, вставку новых строк и обработку обновлений на основе входных данных.

В качестве представления данных в памяти было выбрано строчное хранение, реализованное в классе `TypedDynamicRowTableData`. Такое представление обеспечивает быстрый доступ к строкам таблицы — достаточно вернуть i-тую строку в массиве строк. Операции по работе с обновлением данных также выполняются эффективно: добавление строки происходит путём дописывания значений в конец массива, обновление перезаписывает имеющуюся строку, а удаление затирает строку на заданной позиции.

Поскольку основным сценарием использования динамической таблицы в алгоритме Weever является частое выполнение операций добавления, удаления и обновления строк, строчное представление хорошо соответствует требованиям алгоритма.

При каждом обновлении таблицы удобно иметь интерфейс, позволяющий определить, какие строки были добавлены, удалены или обновлены. Такой интерфейс предоставляют методы `GetInserted`, `GetDeleted` и `GetUpdated`.

1. `GetInserted` — возвращает индексы добавленных строк;

2. `GetDeleted` — возвращает отображение `map<size_t, Row>`, где индексу удаляемого кортежа соответствуют данные, которые были удалены из таблицы.

3. `GetUpdated` — возвращает отображение `map<size_t, Row>`, в котором каждой обновлённой строке соответствует значение до обновления.

Дополнительно классы динамических таблиц теперь поддерживают паттерн стратегии, позволяющий выбирать способ обновления таблицы на лету. На данный момент реализован лишь один способ обновления — `SeparateUpdateStrategy`, основанный на трёх таблицах: первая таблица содержит данные для вставки в конец таблицы, вторая — данные для обновления в формате "индекс обновляемой строки:новые данные", третья — индексы удаляемых строк. Такой подход позволит в будущем без лишних усилий добавить новые способы обновления таблиц. Например, можно указать лишь одну новую таблицу, которая уже содержит добавленные, удалённые или обновлённые строки.

4.3. Индекс

Для эффективного поиска идентификаторов кортежей по значению атрибута при верификации предикатов ЗО в алгоритме используется класс `TableIndex`. Он хранит отдельный индекс для каждого столбца, задействованного хотя бы в одном предикате заданного ЗО, и позволяет за время $O(\log d)$ (d — количество различных значений в `LT-tree`) или $O(1)$ (для атрибутов с предикатами $=, \neq$) находить множества идентификаторов кортежей, удовлетворяющих предикату.

В зависимости от типа оператора предиката для столбца создаётся один из двух видов индексов:

1. `EqualityIndex` — хеш-таблица вида `unordered_map<Component, Roaring>`, для каждого значения атрибута хранится сжатое битовое множество идентификаторов кортежей с таким значением. Используется для предикатов с операторами $=$ и $\neq$.

2. `LessThanIndex` — `LT-tree` с параметром типа `Component`. Используется для предикатов с операторами $<$, $\leq$, $>$, $\geq$ и позволяет эффективно находить все кортежи, значение атрибута которых строго меньше заданного, либо меньше или равно заданному.

Ключом для обоих типов индексов служит класс `Component` — типизированное значение атрибута, инкапсулирующее указатель на зна-

чение (`std::byte const*`) и указатель на объект типа (`model::Type const*`). Класс реализует все операции сравнения, необходимые для работы в `LT-tree`.

Создание и инициализация `TableIndex` происходит следующим образом:

1. При создании `TableIndex` для каждой колонки *col* в предикате ЗО вызывается `CreateIndex(col, op)`, где *op* — тип оператора в предикате. Метод создаёт индекс подходящего типа для данного столбца, если он ещё не существует.

2. Метод `PopulateIndices()` последовательно инициализирует `TableIndex`, вставляя все строки из таблицы в созданные индексы посредством приватного метода `InsertIntoIndices(row_id)`.

3. При каждом обновлении динамической таблицы необходимо обновлять и `TableIndex`, за это отвечает метод `Update()`, который:

   - удаляет из каждого соответствующего индекса записи удалённых из таблицы строк;
   - добавляет записи вставленных строк;
   - для обновлённых строк удаляет старые записи и вставляет новые.

Ниже представлена диаграмма классов `TableIndex` и вспомогательных сущностей:

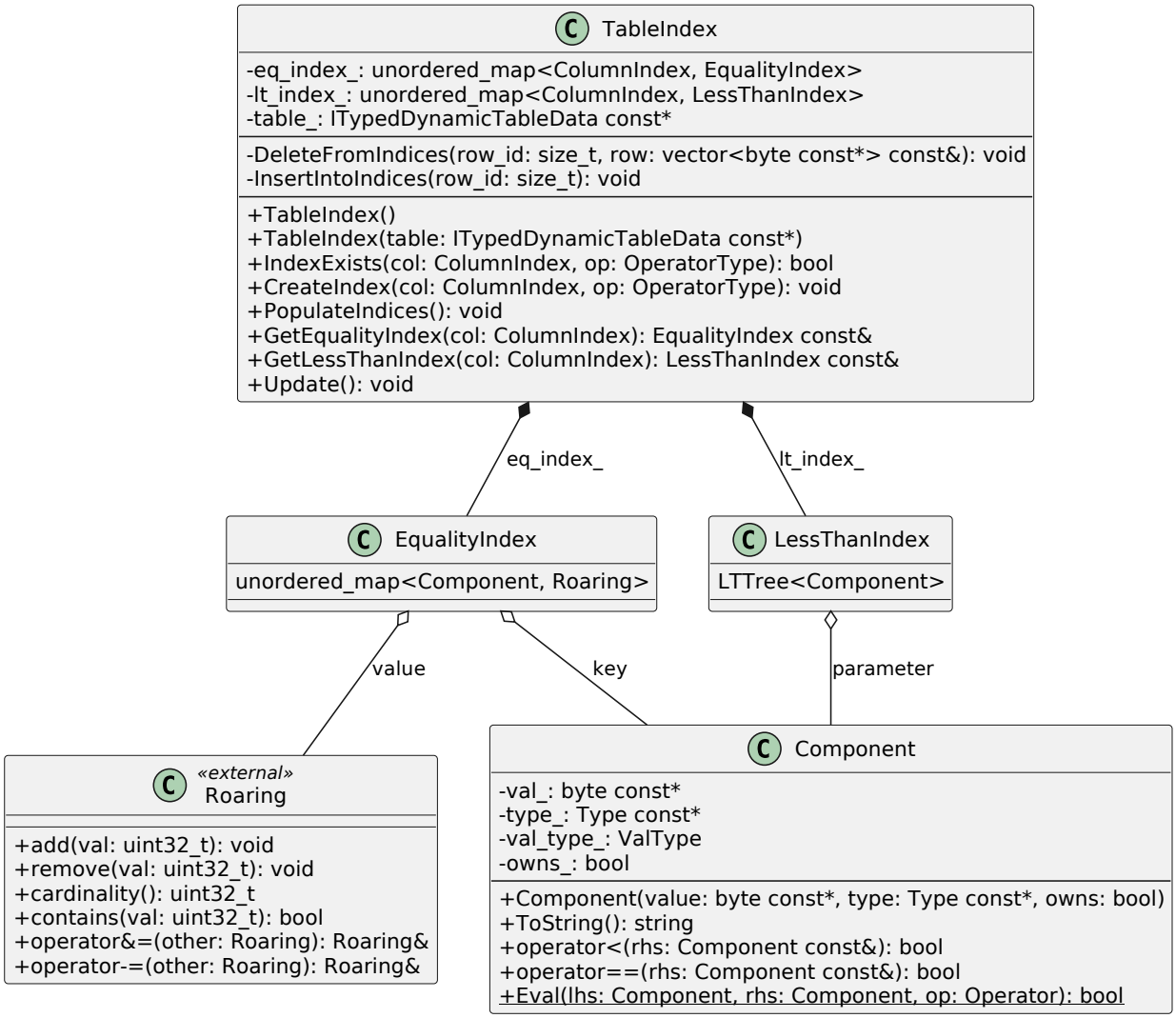


Рис. 4: Диаграмма класса `TableIndex` и вспомогательных классов.

1. `TableIndex` — основной класс, управляющий набором индексов. Предоставляет методы создания, заполнения и обновления индексов, а также доступ к ним по индексу столбца.

2. `EqualityIndex` — псевдоним типа для `unordered_map<Component, Roaring>`. Для каждого уникального значения атрибута хранит сжатое битовое множество идентификаторов кортежей с этим значением.

3. `LessThanIndex` — псевдоним типа для `LTTree<Component>`. Использует `LT-tree` для поддержки диапазонных запросов.

4. `Component` — типизированное значение атрибута. Хранит значение

и его тип, является ключом для обоих видов индексов. Поддерживает операции сравнения и хеширования.

`TableIndex` хранит два отображения: столбец → `EqualityIndex` и столбец → `LessThanIndex`. Если колонка встречается в предикатах с операторами $=$ или $\neq$, то создаётся `EqualityIndex` для этой колонки. Если колонка встречается в предикатах с операторами $<$, $\leq$, $>$, $\geq$, то создаётся `LessThanIndex`.

4.4. Алгоритм `Weever`

Класс `Weever` является основным классом алгоритма и наследуется от базового класса `Algorithm`. Он организует взаимодействие всех описанных выше компонентов и реализует полный цикл верификации: от начальной статической верификации до динамического обновления нарушений при модификации данных.

4.4.1. Отличия от оригинального алгоритма

В оригинальной работе [11] авторы не упоминают, по какому принципу находятся нарушения, необходимые для динамической части алгоритма. Однако их поиск можно осуществлять точно так же, как и в динамическом случае: статический поиск нарушений в исходной таблице эквивалентен последовательной вставке всех её строк в изначально пустое отношение с накоплением обнаруженных нарушений. Аналогичный подход использовался в алгоритме `Facet` [14], где так же применялся конвейерный подход с отсеиванием возможных кандидатов в случае статического набора исходных данных.

Также отметим, что статическую часть можно заменить на любой алгоритм верификации ЗО, поддерживающий поиск нарушений — например, на реализованный в проекте `DCVerifier` [22]. Однако от этого решения было решено отказаться: использование инкрементального подхода для статического анализа избавляет от необходимости поддерживать две независимые реализации поиска нарушений и позволяет

сразу формировать те внутренние структуры данных (индексы, множества нарушений), которые затем используются при обработке обновлений.

Ниже представлена диаграмма класса алгоритма `Weever`, а также вспомогательных классов:

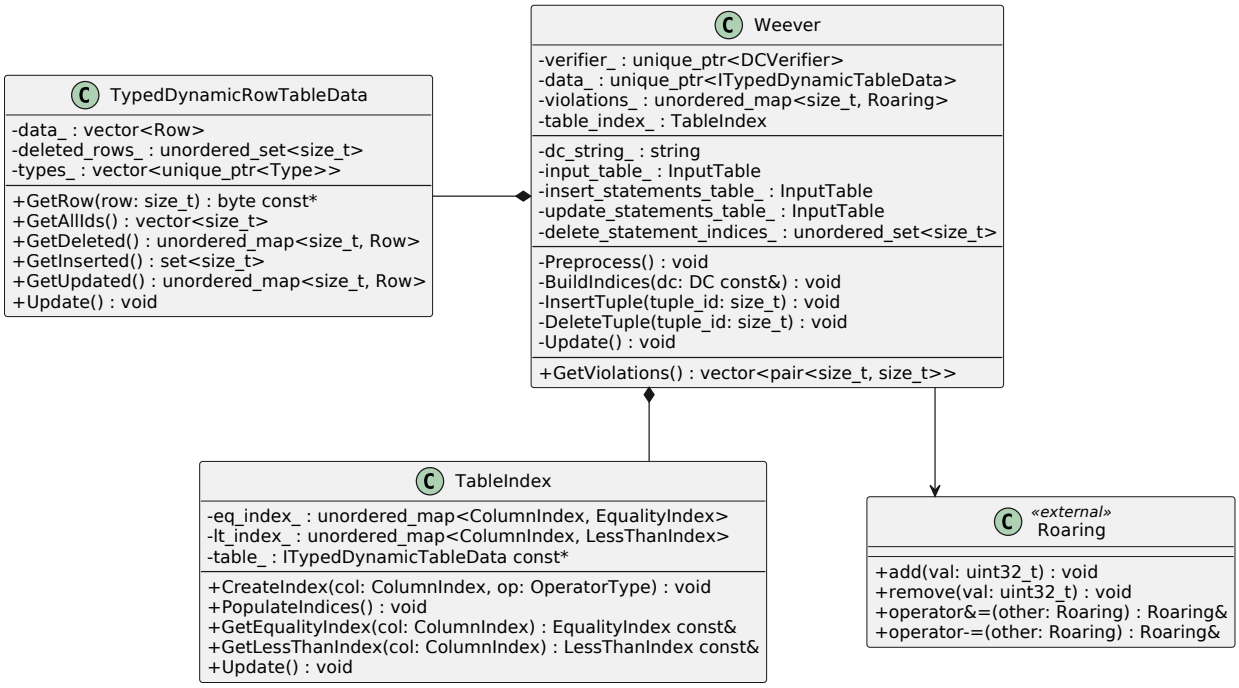


Рис. 5: Диаграмма класса `Weever` и вспомогательных классов.

- `data_` — динамическая типизированная таблица `TypedDynamicRowTableData`;
- `table_index_` — структура `TableIndex` для индексирования таблицы;
- `violations_` — хеш-таблица вида `unordered_map<size_t, Roaring>`, где ключ — идентификатор кортежа, значение — сжатое битовое множество идентификаторов кортежей, с которыми он образует нарушение.

4.4.2. Фаза предобработки

При первом исполнении алгоритма запускается метод `Preprocess()`, последовательно выполняющий следующие шаги:

Построение индексов Для каждого предиката ЗО вызывается `table_index_.CreateIndex(col, op)`, где col — индекс колонки, op — оператор. Метод создаёт подходящий индекс для каждого задействованного столбца. На этом шаге индексы создаются пустыми, лишь выбираются подходящие структуры для индексации колонок.

Последовательная вставка строк. Для каждой строки row_id исходной таблицы вызывается процедура обработки вставки `InsertTuple(row_id)`, после чего строка индексируется в `TableIndex`. Тем самым статический анализ исходной таблицы выполняется средствами самого инкрементального подхода, как было описано выше. К моменту завершения `Preprocess()` накоплено множество всех нарушений ЗО в исходной таблице, а индексы заполнены и готовы к обработке последующих изменений.

4.4.3. Фаза динамического обновления

После применения изменений к таблице, `Weever` вызывает метод `Update()`, обрабатывающий операции в следующем порядке:

1. **Удаление** (`DeleteTuple`): идентификатор каждого удалённого кортежа удаляется из всех битовых множеств в `violations_`, а соответствующее ему множество нарушений очищается.

2. **Вставка** (`InsertTuple`): для каждого вставленного кортежа находятся новые нарушения с уже существующими кортежами и добавляются в `violations_`.

3. **Обновление**: моделируется как удаление старой версии кортежа с последующей вставкой новой.

4. **Синхронизация индексов**: после каждой операции изменения (`InsertTuple`, `DeleteTuple`) вызывается соответствующий метод для обновления данных в индексах.

4.4.4. Обнаружение новых нарушений

Основная логика динамической верификации реализована в методе `Insert`. Для нового кортежа $newT$ формируются два независимых промежуточных множества: s_interm — кандидаты на роль s в паре $(s, newT)$, и t_interm — кандидаты на роль t в паре $(newT, t)$. Оба множества изначально содержат идентификаторы всех существующих кортежей.

Предикаты ЗО обрабатываются последовательно согласно порядку, описанному в разделе 3.3.4. На каждом шаге из индексов извлекается операнд — множество индексов, кандидатов на нарушение:

- для предикатов $=$ и $\neq$ из `EqualityIndex` извлекается множество кортежей с тем же значением атрибута, что у $newT$;
- для предикатов $<$, $>$ из `LessThanIndex` извлекается множество кортежей с меньшим значением атрибута (`FindLess`);
- для предикатов $\leq$, $\geq$ из `LessThanIndex` извлекается множество кортежей со значением, меньшим или равным заданному (`FindLessEqual`).

Промежуточные множества уточняются в зависимости от оператора:

- для $=$, $<$, $\leq$: пересечение ($\cap$) с операндом — оставляем только кортежи, удовлетворяющие предикату;
- для $\neq$, $>$, $\geq$: разность ($\setminus$) с операндом — удаляем кортежи, не удовлетворяющие предикату.

Если оба промежуточных множества оказываются пустыми, обработка прерывается досрочно. По завершении обработки всех предикатов оставшиеся в s_interm кортежи s образуют нарушения $(s,\ newT)$, а кортежи $t \in t_interm$ — нарушения $(newT,\ t)$.

4.5. Python-биндинги

В рамках работы была реализована поддержка использования динамического верификатора в `Python` с использованием библиотеки `pybind11`, что предоставляет пользователю удобный интерфейс взаимодействия с верификатором. Пользователю достаточно передать алгоритму исходные данные, задать ЗО и последовательно передавать алгоритму операции вставки, удаления или обновления. Внутреннее состояние верификатора сохраняется между вызовами, а актуальное множество нарушений доступно через метод `get_violations()`.

5. Эксперименты

5.1. Исследовательские вопросы и метрики

В экспериментах производится сравнение динамического и статического подходов при верификации ЗО. В качестве основной метрики производительности будет рассмотрено время работы алгоритмов. Цель экспериментального исследования — ответить на следующие исследовательские вопросы:

- **RQ1**: Насколько быстрее оказывается инкрементальный подход алгоритма `Weever` относительно статического `DCVerifier`?
- **RQ2**: Какое влияние оказывает количество предикатов и их селективность на производительность инкрементального подхода?

5.2. Условия эксперимента

Характеристики тестового стенда. Эксперименты проводились на тестовом стенде со следующими характеристиками:

- CPU AMD Ryzen 5 3500U @ 2.10GHz x 4 cores (8 threads);
- 8 GiB RAM;
- OS Ubuntu 24.04;
- gcc 13.3.0 (Ubuntu 13.3.0-6ubuntu2 24.04.1), компиляция производилась с опцией -O3.

Методология. Для обеспечения воспроизводимости и стабильности измерений перед запуском тестовых сценариев был проведён ряд настроек системы, минимизирующих влияние посторонних факторов на результаты замеров.

Во-первых, режим управления частотой процессора (CPU governor) был переключён в режим `performance` с помощью утилиты `cpupower`:

```
cpupower frequency-set -g performance
```

Это отключает понижение частоты в моменты низкой нагрузки и обеспечивает работу процессора на стабильно высокой частоте на протяжении всего эксперимента.

Во-вторых, исполняемый процесс был закреплён за одним физическим ядром с помощью утилиты `taskset`, что исключает переключение процесса между ядрами и связанные с ней потери производительности. Одновременно процессу был назначен максимальный приоритет исполнения через `nice`, чтобы минимизировать вытеснение другими процессами системы. Запуск производился следующей командой:

```
sudo nice -n -20 taskset -c 2 ./benchmark
```

Для обеспечения более высокой воспроизводимости перед измерительным замером для каждого эксперимента выполняются два предварительных замера, результаты которых отбрасываются. В качестве итогового значения времени работы для каждой точки используется *медиана* по пяти измерительным прогонам.

5.2.1. Сравнение со статическими подходами

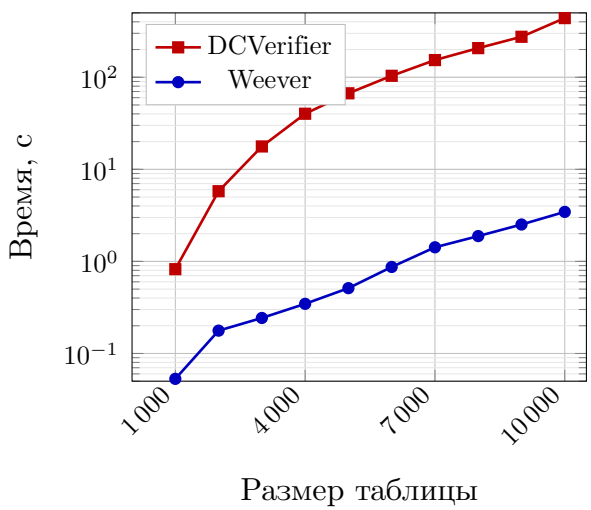
Эксперимент проводился следующим образом: начиная с пустой таблицы, в неё последовательно добавляются записи. После каждой вставки измеряется время инкрементальной обработки алгоритмом `Weever`, после чего на той же таблице запускается статический верификатор `DCVerifier` и измеряется время его работы. Найденные обоими алгоритмами множества нарушений совпали на всех замерах.

В качестве проверяемого ЗО рассматривается следующее ограничение:

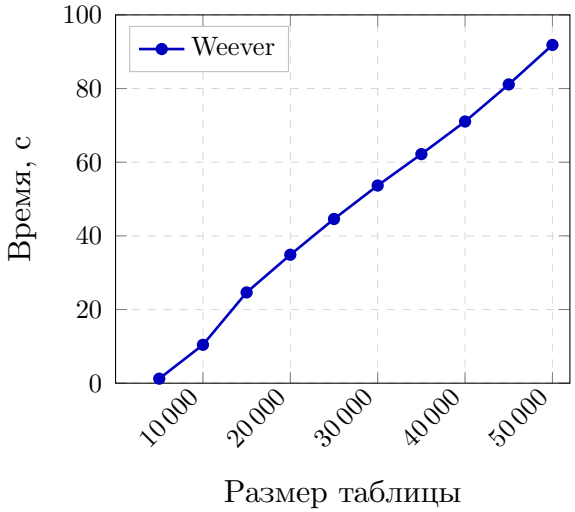
$$\varphi : \forall s, t \in R,\ s \neq t : \neg\big(s.State = t.State \\ \wedge\ s.Salary < t.Salary \\ \wedge\ s.Rate > t.Rate\big)$$

Данные подобраны так, что каждая вставляемая запись образует нарушение со всеми уже присутствующими кортежами. Общее число нарушений в таблице из n записей составляет $\frac{n(n-1)}{2}$ — максимум, возможный для φ. Уже на нескольких десятках тысяч записей оно близится к миллиарду, тем самым моделируется наихудший случай для верификатора.

Эксперимент проводился в двух конфигурациях. В первой размер таблицы варьировался от 1 000 до 10 000 записей с шагом 1 000, и сравнивались оба алгоритма. Во второй — от 5 000 до 50 000 записей с шагом 5 000, здесь измерялась только производительность `Weever`, поскольку `DCVerifier` на таких масштабах становится непрактичным из-за хранения нарушений в виде списка пар индексов, вместо сжатого представления. Результаты приведены на рис. 6.



(a) Сравнение Weever и DCVerifier до 10 000 записей.



(b) Масштабирование Weever до 50 000 записей.

Рис. 6: Зависимость времени работы алгоритмов от размера таблицы.

5.2.2. Обработка динамических операций

Описание эксперимента. Для оценки производительности динамических операций проведены три эксперимента: для операции вставки, обновления и удаления. Тесты проводились на наборе данных *Tax* (один миллион записей, 15 столбцов), для которого заданы три удерживаю-

щихся запрещающих ограничения:

- $\varphi_1 = \neg(s.\text{State} = t.\text{State} \wedge s.\text{Salary} > t.\text{Salary} \wedge s.\text{Rate} < t.\text{Rate})$;
- $\varphi_2 = \neg(s.\text{AreaCode} = t.\text{AreaCode} \wedge s.\text{Phone} = t.\text{Phone} \wedge s.\text{Salary} > t.\text{Salary})$;
- $\varphi_3 = \neg(s.\text{FName} = t.\text{FName} \wedge s.\text{LName} = t.\text{LName} \wedge s.\text{Gender} \neq t.\text{Gender})$.

Эксперименты различаются только типом операции, в каждом из них измеряется время последовательной обработки набора из 1000 строк. Измеряется только время обработки изменяемых строк, предварительная загрузка данных и построение индексов не измеряется. Результаты представлены на рис. 7, 8 и 9.

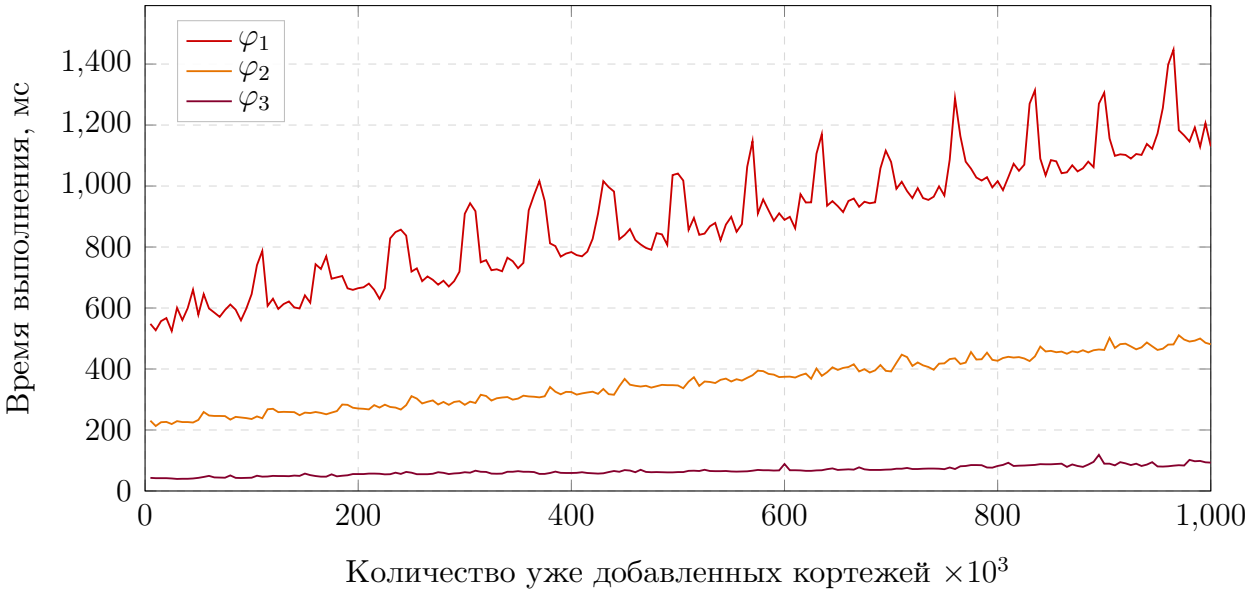


Рис. 7: Время обработки одного набора из 1000 строк при последовательном применении операции *добавления*.

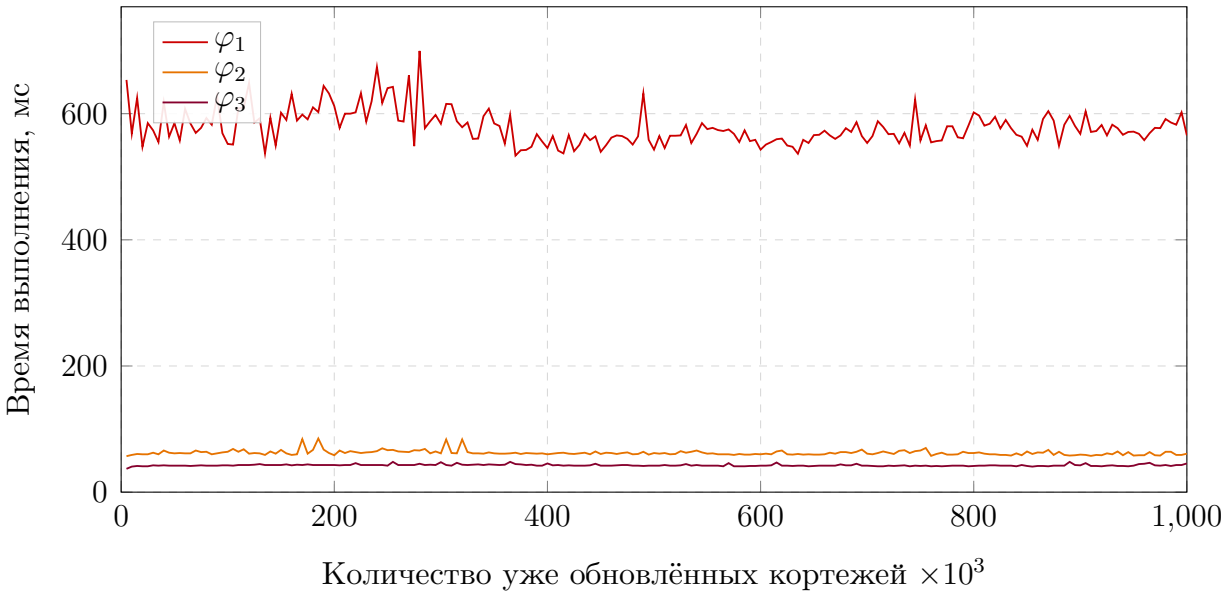


Рис. 8: Время обработки одного набора из 1000 строк при последовательном применении операции *обновления*.

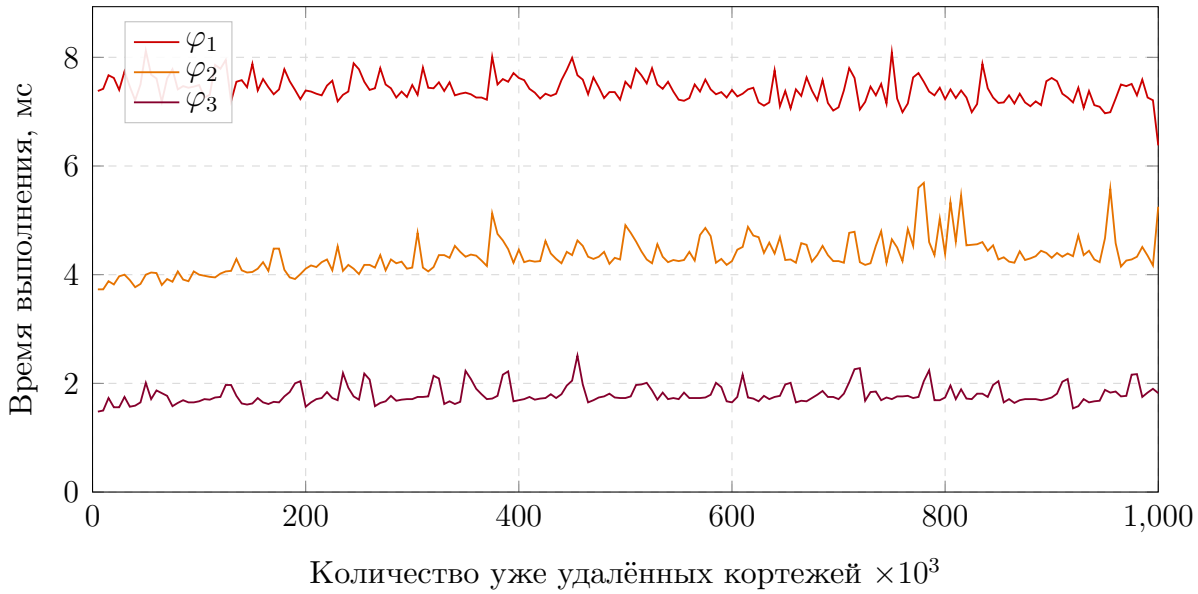


Рис. 9: Время обработки одного набора из 1000 строк при последовательном применении операции *удаления*.

Как можно заметить, время обработки сильно зависит от структуры ЗО, что объясняется устройством алгоритма — предикаты обрабатываются конвейерно, а наиболее дорогостоящими являются предикаты с операторами неравенства, требующие поиска в `LTTree` — для операторов $=, \neq$ обращение к хэш-таблице менее ресурсоёмко.

Вставка (рис. 7) — самая дорогая операция, поскольку каждый новый кортеж необходимо сравнить с уже существующими.

Наибольшее время занимает проверка φ_1, содержащая сразу два предиката с неравенством, из-за чего приходится поддерживать сразу две структуры `LTTree` и выполнять более дорогостоящие операции с промежуточными множествами11. Размер операндов и промежуточных множеств растёт линейно с числом кортежей в таблице, что приводит к линейному росту времени обработки.

Для φ_2 предикаты равенства по `AreaCode` и `Phone` обладают высокой селективностью и быстро сужает множество кандидатов, поэтому неравенство по `Salary` применяется к небольшим по размеру операндам.

Для φ_3 все предикаты являются равенствами или неравенством, поэтому поиск операндом выполняется в хэш-таблицах, что делает обработку почти константной.

Отметим, что на графике присутствуют заметные пики у φ_1, которые связаны с перебалансировкой структур `LTTree` для каждой из колонок в предикатах с неравенством. Также они присутствуют и у φ_2, но менее ярко выражены из-за наличия лишь одного предиката с неравенством.

Обновление (рис. 8), демонстрирует стабильное время работы для всех ЗО, без тенденции роста в течение всего измерения. Однако для φ_1 требуется значительно больше времени, что можно объяснить необходимостью выполнять больше операций с промежуточными множествами, т.к. первый предикат с равенством у φ_1 отсеивает лишь малую часть потенциальных нарушений.

Удаление (рис. 9) во всех случаях обрабатывается на порядки быстрее других операций. Это объясняется тем, что для удаления нет необходимости выполнять дорогостоящие операции с промежуточными множествами: достаточно исключить идентификатор кортежа из индексов и удалить из множества нарушений все пары с его участием. Различие между φ_1, φ_2 и φ_3 почти линейное, что указывает на прямую зависимость от числа предикатов в ЗО.

Заключение

По итогам работы была расширена существующая инфраструктура, а также реализован алгоритм динамической верификации ЗО.

1. Проведён обзор существующих решений;
2. Реализован алгоритм `Weever` для динамической верификации ЗО:
   (a) Реализована поддержка динамического обновления таблиц с выводом типов;
   (b) Реализована структура LT-tree для индексирования динамической таблицы;
3. Сделано возможным использование динамической верификации в `Python`;
4. Проведено экспериментальное исследование алгоритма.

Список литературы

[1] Abedjan Ziawasch, Quiané-Ruiz Jorge-Arnulfo, Naumann Felix. Detecting unique column combinations on dynamic data // Proceedings of the IEEE 30th International Conference on Data Engineering (ICDE). — IEEE, 2014. — P. 1036–1047.

[2] Bian Lingfeng, Yang Weidong et al. An Incremental Algorithm for Repairing Denial Constraint Violations // Information Systems. — 2024.

[3] Boost C++ Libraries. — https://www.boost.org/. — 2026. — Accessed: Мая 25, 2026.

[4] Chu Xu, Ilyas Ihab F., Papotti Paolo. Discovering denial constraints // Proc. VLDB Endow. — 2013. — Aug.. — Vol. 6, no. 13. — P. 1498–1509. — URL: https://doi.org/10.14778/2536258.2536262.

[5] Desbordante: from benchmarking suite to high-performance science-intensive data profiler / George Chernishev, Michael Polyntsov, Anton Chizhov et al. // Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD). — CODS-COMAD '24. — New York, NY, USA : Association for Computing Machinery, 2025. — P. 234–243. — URL: https://doi.org/10.1145/3703323.3703725.

[6] DynFD: Functional Dependency Discovery in Dynamic Datasets / Philipp Schirmer, Thorsten Papenbrock, Sebastian Kruse et al. // International Conference on Extending Database Technology. — 2019. — URL: https://api.semanticscholar.org/CorpusID:81987855.

[7] Fast incremental discovery of pointwise order dependencies / Zijing Tan, Ai Ran, Shuai Ma, Sheng Qin // Proc. VLDB Endow. — 2020. — Jun.. — Vol. 13, no. 10. — P. 1669–1681. — URL: https://doi.org/10.14778/3401960.3401965.

[8] Guibas Leo J., Sedgewick Robert. A dichromatic framework for balanced trees // 19th Annual Symposium on Foundations of Computer Science (sfcs 1978). — 1978. — P. 8–21.

[9] Incremental Discovery of Denial Constraints / Chaoqin Qian, Menglu Li, Zijing Tan et al. // The VLDB Journal. — 2023. — Vol. 32, no. 6. — P. 1289–1313.

[10] Incremental Discovery of Order Dependencies on Tuple Insertions / Lin Zhu, Xu Sun, Zijing Tan et al. // Database Systems for Advanced Applications / Ed. by Guoliang Li, Jun Yang, Joao Gama et al. — Cham : Springer International Publishing, 2019. — P. 157–174.

[11] Kaminsky Youri, Pena Eduardo H. M., Naumann Felix. Incremental Detection of Denial Constraint Violations // Proc. VLDB Endow. — 2024. — Dec.. — Vol. 18, no. 4. — P. 1000–1012. — URL: https://doi.org/10.14778/3717755.3717761.

[12] Lemire Daniel, Ssi-Yan-Kai Gregory, Kaser Owen. Consistently faster and smaller compressed bitmaps with Roaring // Software: Practice and Experience. — 2016. — Vol. 46, no. 11. — P. 1547–1569. — https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.2402.

[13] Pena Eduardo H. M., Porto Fabio, Naumann Felix. Discovering Denial Constraints in Dynamic Datasets // 2024 IEEE 40th International Conference on Data Engineering (ICDE). — 2024. — P. 3546–3558.

[14] Pena Eduardo H. M., de Almeida Eduardo C., Naumann Felix. Fast Detection of Denial Constraint Violations // Proceedings of the VLDB Endowment. — 2021. — Vol. 15, no. 4. — P. 859–871.

[15] Rapidash: Efficient Detection of Constraint Violations / Zifan Liu, Shaleen Deep, Anna Fariha et al. // Proc. VLDB Endow. — 2024. — may. — Vol. 17, no. 8. — P. 2009–2021. — URL: https://doi.org/10.14778/3659437.3659454.

[16] Shaabani Nuhad, Meinel Christoph. Incremental Discovery of Inclusion Dependencies // Proceedings of the 29th International Conference on Scientific and Statistical Database Management. — SSDBM '17. — New York, NY, USA : Association for Computing Machinery, 2017. — 12 p. — URL: https://doi.org/10.1145/3085504.3085506.

[17] The Standard Template Library : Rep. : HPL-94-34 / Hewlett-Packard Laboratories ; Executor: Alexander A. Stepanov, Meng Lee : 1994.

[18] Wang Shyue-Liang, Shen Ju-Wen, Hong Tzung-Pei. Incremental discovery of functional dependencies using partitions // Proceedings Joint 9th IFSA World Congress and 20th NAFIPS International Conference (Cat. No. 01TH8569). — Vol. 3. — 2001. — P. 1322–1326 vol.3.

[19] Адельсон-Вельский Г. М., Ландис Е. М. Один алгоритм организации информации // Доклады АН СССР. — 1962. — Vol. 146, no. 2. — P. 263–266.

[20] Павел Аносов. Верификация DC в Desbordante. — 2024. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/DC%20Verification%20-%20Anosov%20Pavel%20-%202024%20spring.pdf (дата обращения: 2025-11-09).

[21] Павел Аносов. Реализация алгоритма верификации приближенных запрещающих ограничений в проекте Desbordante. — 2024. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/ADC%20Verification%20And%20Metrics%20Survey%20-%20Anosov%20Pavel%20-%202025%20spring.pdf (дата обращения: 2025-11-09).

[22] Павел Аносов. Улучшение алгоритма верификации запрещающих ограничений в Desbordante. — 2024. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/Constant%20DC%20Verification%20and%20Minimization%20-%20Anosov%20Pavel%20-%202024%20autumn.pdf (дата обращения: 2025-11-09).